

Práctica 4:

Control remoto de la Raspberry Pi via WiFi y servidor web

Fundamentos de Sistemas Embebidos

Autor: José Mauricio Matamoros de Maria y Campos

Versión: 2026.03

1. Objetivo

El alumno aprenderá a configurar la Raspberry Pi como punto de acceso inalámbrico que permita acceder a un servidor web simple que controle el puerto GPIO de la misma.

2. Material

Se asume que el alumno cuenta con una Raspberry Pi con sistema operativo Raspberry Pi OS e interprete de Python instalado. Se aconseja encarecidamente el uso de *git* como programa de control de versiones.

Si se cuenta con una Raspberry Pi sin WiFi integrado (e.j Raspberry Pi2), se precisará de un adaptador WiFi USB compatible para la misma.

Además, el alumno necesitará el alambrado de la [Práctica 3](#).

3. Instrucciones

1. Alambre el circuito tal y como se detalla en la [Práctica 3](#).
2. Realice los ejercicios y experimentos de la [Práctica 3](#).
3. Configure la Raspberry Pi como punto de acceso inalámbrico.
4. Levante un servidor de prueba como se detalla en la [Subsección 3.3](#).
5. Realice los experimentos propuestos en la [Sección 4](#).

Raspberry Pi OS: Trixie

Esta práctica requiere un sistema operativo Raspberry Pi OS basado en Debian 13 (Trixie)
Para verificar que cuenta con la versión adecuada ejecute el comando `lsb_release -a`

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Debian
Description:    Debian GNU/Linux 13 (trixie)
Release:        13
Codename:       trixie
```

3.1. Paso 1: Instalación de paquetes

Instale los paquetes `DNSMasq` y `python3-magic` tras actualizar su Pi:

```
# apt-get update
# apt-get upgrade
# apt-get install python3-magic
```

O bien, mediante pip

```
$ pip3 install -U python-magic
```

3.2. Paso 2: Configuración de la Raspberry Pi como punto de acceso inalámbrico

Para operar como punto de acceso la Raspberry Pi necesita tener instalado el software apropiado, incluyendo un servidor DHCP para proporcionar a los dispositivos que se conecten una dirección IP.

eth0

En esta práctica se modificará la configuración del adaptador inalámbrico, por lo que no podrá usarlo para conectarse a internet.

Antes de continuar, conecte su Raspberry Pi a una red cableada mediante el adaptador `eth0` tal como se indica en el manual de la Práctica 1 o configurándola con NetworkManager usando `nmtui-edit`.

Para configurar una red independiente con servidor DHCP la Raspberry Pi debe tener asignada una dirección IP estática en el adaptador inalámbrico que proveerá la conexión. Debido a que la Raspberry Pi tiene un procesador pequeño, se configurará para servir en una red privada clase C, es decir con direcciones IP del tipo `192.168.X.Y`. Así mismo, se supondrá que el dispositivo inalámbrico utilizado es `wlan0`.

3.2.1. Configuración del punto de acceso con DHCP

Para configurar el punto de acceso se debe crear una nueva conexión tipo punto de acceso (*hotspot*) con NetworkManager. Esto se logra con el siguiente comando:

```
$ nmcli connection add con-name AccessPoint type wifi ifname wlan0 ssid RaspberryAP \
  802-11-wireless.mode ap 802-11-wireless.band bg wifi-sec.pairwise ccmp wifi-sec.proto rsn \
  wifi-sec.key-mgmt wpa-psk wifi-sec.psk 12345678 ipv6.method disabled ipv4.method shared \
  ipv4.address 192.168.1.254/24 ipv4.gateway 192.168.1.254 autoconnect yes
```

donde:

- AccessPoint es el nombre de la conexión como será conocida por NetworkManager
- RaspberryAP es el SSID de la red
- bg son los modos de operación
- 12345678 es la contraseña para la conexión
- 192.168.1.254 es la dirección IP asignada a la interfaz.

La configuración ingresada configura la Raspberry Pi para crear una red inalámbrica tipo 802.11b/g con el nombre *RaspberryAP* y contraseña 12345678. La seguridad de forma predeterminada es WPA2.

Los modos de operación posibles son:

- a = IEEE 802.11a (5 GHz)
- b = IEEE 802.11b (2.4 GHz)
- g = IEEE 802.11g (2.4 GHz)

Importante: Tanto el nombre de la red o SSID y la contraseña no deben entrecruzarse. La contraseña debe tener entre 8 y 64 caracteres. **Cambie el SSID a Raspberry_Apellido para evitar conflictos.** Puede hacerlo con `nmtui-edit`, `nmcli connection modify`, o bien editando el archivo `.nmconnection` asociado.

Ahora revise el archivo `/etc/NetworkManager/system-connections/AccessPoint.nmconnection`. Deberá ver algo similar a lo siguiente.

```

1 [connection]
2 id=AccessPoint
3 uuid=ebf75b60-23be-4b65-9227-29b21ebadb00
4 type=wifi
5 interface-name=wlan0
6
7 [wifi]
8 band=bg
9 mode=ap
10 ssid=RaspberryAP
11
12 [wifi-security]
13 group=ccmp;
14 key-mgmt=wpa-psk
15 pairwise=ccmp;
16 proto=rsn;
17 psk=12345678
18
19 [ipv4]
20 address1=192.168.1.254/24
21 gateway=192.168.1.254
22 may-fail=false
23 method=shared
24
25 [ipv6]
26 addr-gen-mode=default
27 method=disabled
28
29 [proxy]
30

```

3.2.2. Habilitación del punto de acceso

NetworkManager no habilita el punto de acceso de forma predeterminada, por lo que es necesario activarlo manualmente. Además, precisaremos de reiniciar antes el servicio para que recargue las configuraciones ingresadas y habilite dnsmasq. Para habilitar el punto de acceso ejecute el siguiente comando:

```

# systemctl restart NetworkManager
$ nmcli connection up AccessPoint

```

Verifique que la configuración de la conexión y que el adaptador está habilitado con la IP configurada.

```

$ nmcli connection show AccessPoint
$ ifconfig

```

Como prueba de fuego, intente conectarse a la Raspberry Pi desde su dispositivo móvil usando el SSID y la contraseña ingresados. La conexión debería ser exitosa aunque notará que no cuenta con conexión a internet.

3.2.3. Habilitación de activación automática del punto de acceso

De igual manera, las conexiones de NetworkManager no inician automáticamente. Para superar este inconveniente es necesario instruir a NetworkManager para que lance el punto de acceso de forma automática tras cada reinicio. Esto se logra con la siguiente instrucción:

```

$ nmcli connection modify connection.autoconnect yes connection.autoconnect-priority 1

```

Finalmente reinicie su Raspberry Pi y verifique la activación automática del punto de acceso.

3.3. Paso 2: Configuración de la Raspberry Pi como servidor Web

Raspbian es una variante de Debian, por lo que se le dará bien servir páginas web de forma segura, especialmente cuando se utiliza Apache. Sin embargo, configurar Apache para enlazarse con Python y operar la GPIO no es una tarea trivial, por lo que en esta práctica se utilizará un servidor web simple basado en el `BaseHTTPRequestHandler` que incorpora el paquete `http.server` de Python.

Para habilitar un servidor web en Python, basta con heredar de la clase `BaseHTTPRequestHandler` e implementar el método `do_GET` para que imprima el código HTML al socket vía el método `self.wfile.write` tal como se muestra en el [Código de Ejemplo 1](#).

Código ejemplo 1: Archivo `simple-webserver.py`

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2
3 class WebServer(BaseHTTPRequestHandler):
4     def do_GET(self):
5         self.send_response(200)
6         self.send_header("Content-type", "text/html")
7         self.end_headers()
8         self.wfile.write(bytes("<html><body>Hola Mundo!!!</body></html>", "utf-8"))
9
10 def main():
11     webServer = HTTPServer(("192.168.1.254", 80), WebServer)
12     print("Servidor iniciado")
13     print("\tAtendiendo solicitudes entrantes")
14     try:
15         webServer.serve_forever()
16     except KeyboardInterrupt:
17         pass
18     webServer.server_close()
19     print("Server stopped.")
20
21 # Punto de anclaje de la función main
22 if __name__ == "__main__":
23     main()
```

Script de Python presentado inicia un servidor web que atiende todas las peticiones entrantes vía la interfaz con la IP 192.168.1.254 (el punto de acceso) en el puerto 80 (HTTP predeterminado). A cada petición se le devolverá una señal de estado HTTP200 u *OK*, seguido por código HTML. Es importante aclarar que para cada archivo servido se debe especificar el tipo de archivo en la cabecera.

Importante: El puerto 80 (y en general todos los puertos por debajo del 2014) están reservados para servicios de sistema, por lo que Python fallará al intentar levantar el servidor web en este puerto. Existen dos opciones: puede ejecutar el proceso como superusuario con `sudo` o bien usar otro puerto como el 8080.

Genere el archivo `simple-webserver.py` y ejecútelo. A continuación, conéctese a la Raspberry Pi con cualquier dispositivo móvil e ingrese a la dirección IP del punto de acceso, es decir: <http://192.168.1.254>.

Con ligeras modificaciones es posible servir cualquier tipo de archivo. Todas las peticiones ingresadas en la barra de direcciones del navegador llegarán por método *GET*, por lo que deberán ser procesadas en el método `do_GET`, accediendo al atributo de clase `self.path`, relativo al directorio de trabajo. En caso de que no se proporcione un archivo, `do_GET` tendrá que proporcionar la página por defecto, típicamente nombrada `index.html`, pero que en este caso por motivos didácticos se ha nombrado `user_interface.html` (véase [Código de Ejemplo 2](#)).

Código ejemplo 2: Método do_GET del archivo webserver.py

```
1 def do_GET(self):
2     # Revisamos si se accede a la raiz.
3     # En ese caso se responde con la interfaz por defecto
4     if self.path == '/':
5         # 200 es el código de respuesta satisfactorio (OK)
6         # de una solicitud
7         self.send_response(200)
8         # La cabecera HTTP siempre debe contener el tipo de datos mime
9         # del contenido con el que responde el servidor
10        self.send_header("Content-type", "text/html")
11        # Fin de cabecera
12        self.end_headers()
13        # Por simplicidad, se devuelve como respuesta el contenido del
14        # archivo html con el código de la página de interfaz de usuario
15        self._serve_ui_file()
16        # En caso contrario, se verifica que el archivo exista y se sirve
17    else:
18        self._serve_file(self.path[1:])
```

Para servir un archivo se tiene que verificar que el éste exista, proporcionar su tipo mime en la cabecera y devolver los datos como una cadena binaria. Esto se realiza en el método interno `_serve_file`. Si el archivo no se encontrare, se devuelve un error *HTTP404* como se muestra en el [Código de Ejemplo 3](#):

Código ejemplo 3: Método _serve_file del archivo webserver.py

```
1 def _serve_file(self, rel_path):
2     if not os.path.isfile(rel_path):
3         self.send_error(404)
4         return
5     self.send_response(200)
6     mime = magic.Magic(mime=True)
7     self.send_header("Content-type", mime.from_file(rel_path))
8     self.end_headers()
9     with open(rel_path, 'rb') as file:
10        self.wfile.write(file.read())
```

La interacción cliente servidor se lleva a cabo de manera similar. Dependerá de si los datos se envían por método *GET* o *POST*, de los cuales se prefiere el segundo pues hace más difícil inyectar datos. De manera análoga se utiliza el método `do_POST` que recibe y procesa los datos. En esta práctica, se utilizan datos codificados mediante JSON para hacer llamadas asíncronas del cliente y sin respuesta por parte del servidor (véase [Código de Ejemplo 4](#)).

Código ejemplo 4: Método `do_POST` del archivo `webserver.py`

```
1 def do_POST(self):
2     # Primero se obtiene la longitud de la cadena de datos recibida
3     content_length = int(self.headers.get('Content-Length'))
4     if content_length < 1:
5         return
6     # Después se lee toda la cadena de datos
7     post_data = self.rfile.read(content_length)
8     # Finalmente, se decodifica el objeto JSON y se procesan los datos.
9     # Se descartan cadenas de datos mal formados
10    try:
11        jobj = json.loads(post_data.decode("utf-8"))
12        self._parse_post(jobj)
13    except:
14        print(sys.exc_info())
15        print("Datos POST no reconocidos")
```

El método `do_POST` preentado en el [Código de Ejemplo 4](#) interpreta los datos recibidos como cadenas de texto unicode de 8 bits (*utf-8*) que contienen objetos en JSON que son decodificados a un diccionario de Python. El diccionario es después enviado al método interno `_parse_post` mostrado en el [Código de Ejemplo 5](#) que analiza los datos y realiza las acciones pertinentes.

Código ejemplo 5: Método `_parse_post` del archivo `webserver.py`

```
1 def _parse_post(self, json_obj):
2     if not 'action' in json_obj or not 'value' in json_obj:
3         return
4     switcher = {
5         'led' : leds,
6         'marquee' : marquee,
7         'numpad' : bcd
8     }
9     func = switcher.get(json_obj['action'], None)
10    if func:
11        print('\tCall{{{}}}'.format(func, json_obj['value']))
12        func(json_obj['value'])
```

Genere los archivos `webserver.py` y `user_interface.html` (véase [Apéndices A](#) y [B](#)), luego ejecute el script de Python. A continuación, conéctese a la Raspberry Pi con cualquier dispositivo móvil e ingrese a la dirección IP del punto de acceso, es decir: <http://192.168.1.254:8080>. Debería ver una pantalla similar a la siguiente.

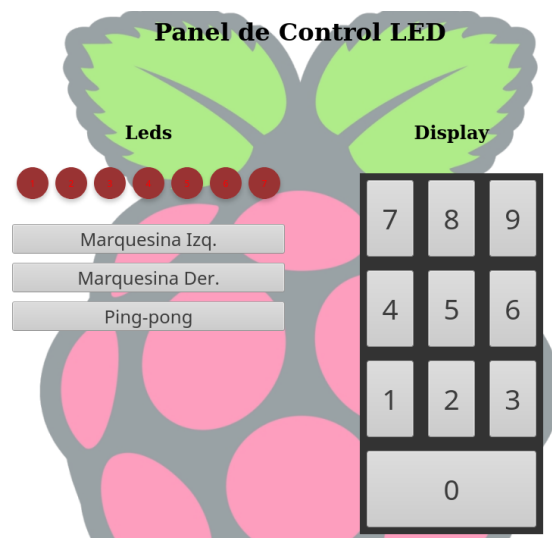


Figura 1: Caption: Intefaz de usuario del controlador de Leds en la Raspberry Pi.

4. Experimentos

Integre el código de la [Práctica 3](#) en un archivo python llamado `led_manager.py` y que ofrezca las siguientes funciones:

1. [2 pts] Encendido del del 1–7 al presionar el boton adecuado
2. [2 pts] Desplegado de la marquesina izquierda al presionar el boton adecuado
3. [2 pts] Desplegado de la marquesina derecha al presionar el boton adecuado
4. [2 pts] Desplegado de la marquesina tipo ping-pong al presionar el boton adecuado
5. [2 pts] Desplegado del dígito correcto en el display de 7 segmentos al presionar el boton correspondiente

Evidencias: Entregue sólo UN video corto donde se observe el funcionamiento del sistema operado desde su teléfono móvil o tableta.

Nota 1: No es aceptable controlar el sistema desde el navegador en la misma Raspberry Pi (`127.0.0.1` o `localhost`).

Nota 2: Las marquesinas deberán ejecutarse *ad infinitum*, interrumpiéndose sólo cuando se selecciona el encendido de un botón en particular.

Experimentos opcionales:

1. [+2 pts] Modifique la implementación presentada para que la interfaz web refleje el estado de la Pi en todo momento, incluyendo la animación de las marquesinas.
2. [+1 pts] Modifique la implementación presentada para que sea posible definir la velocidad de rotación de la marquesina en la interfaz web.
3. [+2 pts] Genere un único script de *shell* (ej. *bash*) que automatice la configuración del punto de acceso inalámbrico. Dicho script recibirá dos parámetros opcionales: el SSID de la red y la contraseña a utilizar.

A. El archivo `webserver.py`

Código ejemplo 6: Archivo `webserver.py`

```
1 import os
2 import sys
3 import json
4 import magic
5 from led_manager import leds, bcd, marquee
6 from http.server import BaseHTTPRequestHandler,
  HTTPServer
7 # import time
8 # import time
9
10 # Nombre o dirección IP del sistema anfitrión del
  servidor web
11 # address = "localhost"
12 address = "192.168.1.254"
13 # Puerto en el cual el servidor estará atendiendo
  solicitudes HTTP
14 # El default de un servidor web en producción debe ser 80
15 port = 8080
16
17
18 class WebServer(BaseHTTPRequestHandler):
19     """Sirve cualquier archivo encontrado en el servidor
20     """
21     def _serve_file(self, rel_path):
22         if not os.path.isfile(rel_path):
23             self.send_error(404)
24             return
25         self.send_response(200)
26         mime = magic.Magic(mime=True)
27         self.send_header("Content-type", mime.from_file(
28             rel_path))
29         self.end_headers()
30         with open(rel_path, 'rb') as file:
31             self.wfile.write(file.read())
32
33     """Sirve el archivo de interfaz de usuario"""
34     def _serve_ui_file(self):
35         if not os.path.isfile("user_interface.html"):
36             err = "user_interface.html not found."
37             self.wfile.write(bytes(err, "utf-8"))
38             print(err)
39             return
40         try:
41             with open("user_interface.html", "r") as f:
42                 content = "\n".join(f.readlines())
43         except:
44             content = "Error reading user_interface.html"
45             self.wfile.write(bytes(content, "utf-8"))
46
47     def _parse_post(self, json_obj):
48         if not 'action' in json_obj or not 'value' in
49             json_obj:
50             return
51         switcher = {
52             'led': leds,
53             'marquee': marquee,
54             'numpad': bcd
55         }
56         func = switcher.get(json_obj['action'], None)
57         if func:
58             print('\tCall{ }({})'.format(func, json_obj['value']
59             ))
60             func(json_obj['value'])
61
62     """do_GET controla todas las solicitudes recibidas vía
63     GET, es
64     decir, páginas. Por seguridad, no se analizan
65     variables que lleguen
66     por esta vía"""
67     def do_GET(self):
68         # Revisamos si se accede a la raíz.
69         # En ese caso se responde con la interfaz por
70         defecto
71
72     if self.path == '/':
73         # 200 es el código de respuesta satisfactorio (OK)
74         # de una solicitud
75         self.send_response(200)
76         # La cabecera HTTP siempre debe contener el tipo
77         de datos mime
78         # del contenido con el que responde el servidor
79         self.send_header("Content-type", "text/html")
80         # Fin de cabecera
81         self.end_headers()
82         # Por simplicidad, se devuelve como respuesta el
83         contenido del
84         # archivo html con el código de la página de
85         interfaz de usuario
86         self._serve_ui_file()
87         # En caso contrario, se verifica que el archivo
88         exista y se sirve
89     else:
90         self._serve_file(self.path[1:])
91
92     """do_POST controla todas las solicitudes recibidas vía
93     a POST, es
94     decir, envíos de formulario. Aquí se gestionan los
95     comandos para
96     la Raspberry Pi"""
97     def do_POST(self):
98         # Primero se obtiene la longitud de la cadena de
99         datos recibida
100         content_length = int(self.headers.get('Content-
101             Length'))
102         if content_length < 1:
103             return
104         # Después se lee toda la cadena de datos
105         post_data = self.rfile.read(content_length)
106         # Finalmente, se decodifica el objeto JSON y se
107         procesan los datos.
108         # Se descartan cadenas de datos mal formados
109         try:
110             jobj = json.loads(post_data.decode("utf-8"))
111             self._parse_post(jobj)
112         except:
113             print(sys.exc_info())
114             print("Datos POST no reconocidos")
115
116 def main():
117     # Inicializa una nueva instancia de HTTPServer con el
118     # BaseHTTPRequestHandler definido en este archivo
119     webServer = HTTPServer((address, port), WebServer)
120     print("Servidor iniciado")
121     print("\tAtendiendo solicitudes en http://{ }:{ }".
122         format(
123             address, port))
124
125     try:
126         # Mantiene al servidor web ejecutándose en segundo
127         plano
128         webServer.serve_forever()
129     except KeyboardInterrupt:
130         # Maneja la interrupción de cierre CTRL+C
131         pass
132     except:
133         print(sys.exc_info())
134         # Detiene el servidor web cerrando todas las
135         conexiones
136         webServer.server_close()
137         # Reporta parada del servidor web en consola
138         print("Server stopped.")
139
140 # Punto de anclaje de la función main
141 if __name__ == "__main__":
142     main()
```

B. El archivo user_interface.html

Código ejemplo 7: Archivo user_interface.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Panel de Control LED - Raspberry Pi</title>
5 <meta charset="ISO-8859-1">
6 <style type="text/css">
7   html{
8     width: 100vw;
9     height: 100vh;
10    min-width: 100vw;
11    min-height: 100vh;
12    margin: 0;
13    padding: 0;
14    box-sizing: border-box;
15    overflow: hidden;
16  }
17
18  body{
19    width: 800px;
20    height: 100vh;
21    max-width: 800px;
22    min-height: 100vh;
23    padding: 0;
24    margin: 0 auto;
25    box-sizing: border-box;
26  }
27
28  body::after{
29    content: "";
30    position: absolute;
31    top: 0;
32    left: 0;
33    bottom: 0;
34    right: 0;
35    z-index: -1;
36    opacity: 0.5;
37    background-image: url('img/raspberry.png');
38    background-repeat: no-repeat;
39    background-attachment: fixed;
40    background-position: center;
41    background-size: contain;
42  }
43
44  header{
45    width: 100%;
46    padding: 0;
47    margin: 0;
48    box-sizing: border-box;
49    text-align: center;
50  }
51
52  h1{
53    height: 2em;
54    padding: 1em 0;
55  }
56
57  .container{
58    width: 100%;
59    padding: 0;
60    margin: 0;
61    box-sizing: border-box;
62    display: flex;
63    flex-direction: row;
64  }
65
66  .column{
67    flex: 1 0 0;
68    display: flex;
69    flex-direction: column;
70  }
71
72  .numpad{
73    width: 6.5em;
74    height: 12.75em;
75    background-color: #333333;
76    border-radius: 5px;
77    margin: 0.5em auto;
78    padding: 0.2em;
79    font-size: 28pt;
80    display: grid;
81    grid-template-columns: auto auto auto;
82  }
83
84  .numbutton{
85    font-size: inherit;
86    flex: 1 0 0;
87    margin: 0.125em;
88  }
89
90  .ledstrip{
91    justify-content: space-evenly;
92    width: 90%;
93    height: 4em;
94    padding: 0;
95    margin: 0.5em auto;
96    /*background-color: #333333;*/
97  }
98
99  .ledbutton{
100   width: 3.5em;
101   height: 3.5em;
102   color: #F00;
103   background-color: #933;
104   border-radius: 50%;
105   margin: 0.25em;
106   padding: 0;
107   border: none;
108   box-shadow: 0px 4px 5px rgba(0, 0, 0, 0.2);
109 }
110
111
112 .ledbutton:hover{
113   background-color: #F33;
114 }
115
116 .widebutton{
117   font-size: 18pt;
118   width: 90%;
119   margin: 0.25em auto;
120 }
121
122 .on{
123   color: #0F0;
124   background-color: #3F3;
125 }
126 </style>
127 </head>
128 <body>
129 <header><h1>Panel de Control LED</h1></header>
130 <section class="container">
131   <article class="column">
132     <header><h2>Leds</h2></header>
133     <section class="container ledstrip">
134       <button class="ledbutton" onclick="handle(this, 'led', 1)">1</button>
135       <button class="ledbutton" onclick="handle(this, 'led', 2)">2</button>
136       <button class="ledbutton" onclick="handle(this, 'led', 3)">3</button>
137       <button class="ledbutton" onclick="handle(this, 'led', 4)">4</button>
138       <button class="ledbutton" onclick="handle(this, 'led', 5)">5</button>
139       <button class="ledbutton" onclick="handle(this, 'led', 6)">6</button>
140       <button class="ledbutton" onclick="handle(this, 'led', 7)">7</button>
141     </section>
142     <button class="widebutton" onclick="handle(this, 'marquee', 'left')">Marquesina Izq.</button>
143   </article>
144 </section>
145 </body>
146 </html>
```

```

143     <button class="widebutton" onclick="handle(this, '
144     marquee', 'right')">Marquesina Der.</button>
145     <button class="widebutton" onclick="handle(this, '
146     marquee', 'pingpong')">Ping-pong</button>
147 </article>
148 <article class="column">
149     <header><h2>Display</h2></header>
150     <section class="container numpad">
151         <button class="numbutton" onclick="handle(this,
152         'numpad', 7)">7</button>
153         <button class="numbutton" onclick="handle(this,
154         'numpad', 8)">8</button>
155         <button class="numbutton" onclick="handle(this,
156         'numpad', 9)">9</button>
157         <button class="numbutton" onclick="handle(this,
158         'numpad', 4)">4</button>
159         <button class="numbutton" onclick="handle(this,
160         'numpad', 5)">5</button>
161         <button class="numbutton" onclick="handle(this,
162         'numpad', 6)">6</button>
163         <button class="numbutton" onclick="handle(this,
164         'numpad', 1)">1</button>
165         <button class="numbutton" onclick="handle(this,
166         'numpad', 2)">2</button>
167         <button class="numbutton" onclick="handle(this,
168         'numpad', 3)">3</button>
169         <div class="numbutton" ></div>
170         <button class="numbutton" onclick="handle(this,
171         'numpad', 0)">0</button>
172         <div class="numbutton" ></div>
173     </section>
174 </article>
175 </section>
176 </body>
177
178 </html>
179 <script language="javascript">
180 <!--
181 function deactivateAll(){
182     var buttons = document.getElementsByTagName('button');
183     for(button in buttons)
184         button.classList.remove("on")
185 }
186
187 function activate(sender){
188     if(sender == null)
189         return;
190     sender.classList.add("on");
191 }
192
193 function handle(sender, action, value){
194     // deactivateAll();
195     // activate(sender);
196     submit(action, value);
197 }
198
199 function submit(action, value){
200     var xhr = new XMLHttpRequest();
201     xhr.open("POST", window.location.href, true);
202     xhr.setRequestHeader('Content-Type', 'application/json
203     ');
204     xhr.send(JSON.stringify({
205         'action' : action,
206         'value' : value,
207     }));
208 }
209 //-->
210 </script>

```